BANQUE CENTRALE DE MAURITANIE

Document de démonstration technique

Application de gestion des prêts de matériel informatique

Stage Licence 2 — Développement d'une API REST Spring Boot
avec intégration PostgreSQL et interface web React

Octobre 2026

Ce document présente, avec résultats réels à l'appui, l'ensemble du travail réalisé : architecture backend, API REST, règles métier, intégration avec la base de données PostgreSQL, interface frontend connectée à l'API, jeu de données de démonstration et résultats des tests.

1. Vue d'ensemble du projet

L'application permet de gérer le cycle complet de prêt de matériel informatique (ordinateurs portables, écrans, téléphones, vidéoprojecteurs, disques externes) aux employés de la BCM : inventaire, prêt, restitution, détection automatique des retards et historique.

Composant	Technologie	Statut
Backend / API REST	Java 21, Spring Boot 4.1.1, Spring Data JPA, Jakarta Validation	Fonctionnel, testé en direct
Base de données	PostgreSQL 18 (Postgres.app)	Connectée, schéma généré automatiquement
Frontend	React 19, TypeScript, Vite, Tailwind CSS	Fonctionnel, connecté à l'API
Tests	JUnit 5, Mockito	34 tests, 0 échec

2. Architecture backend

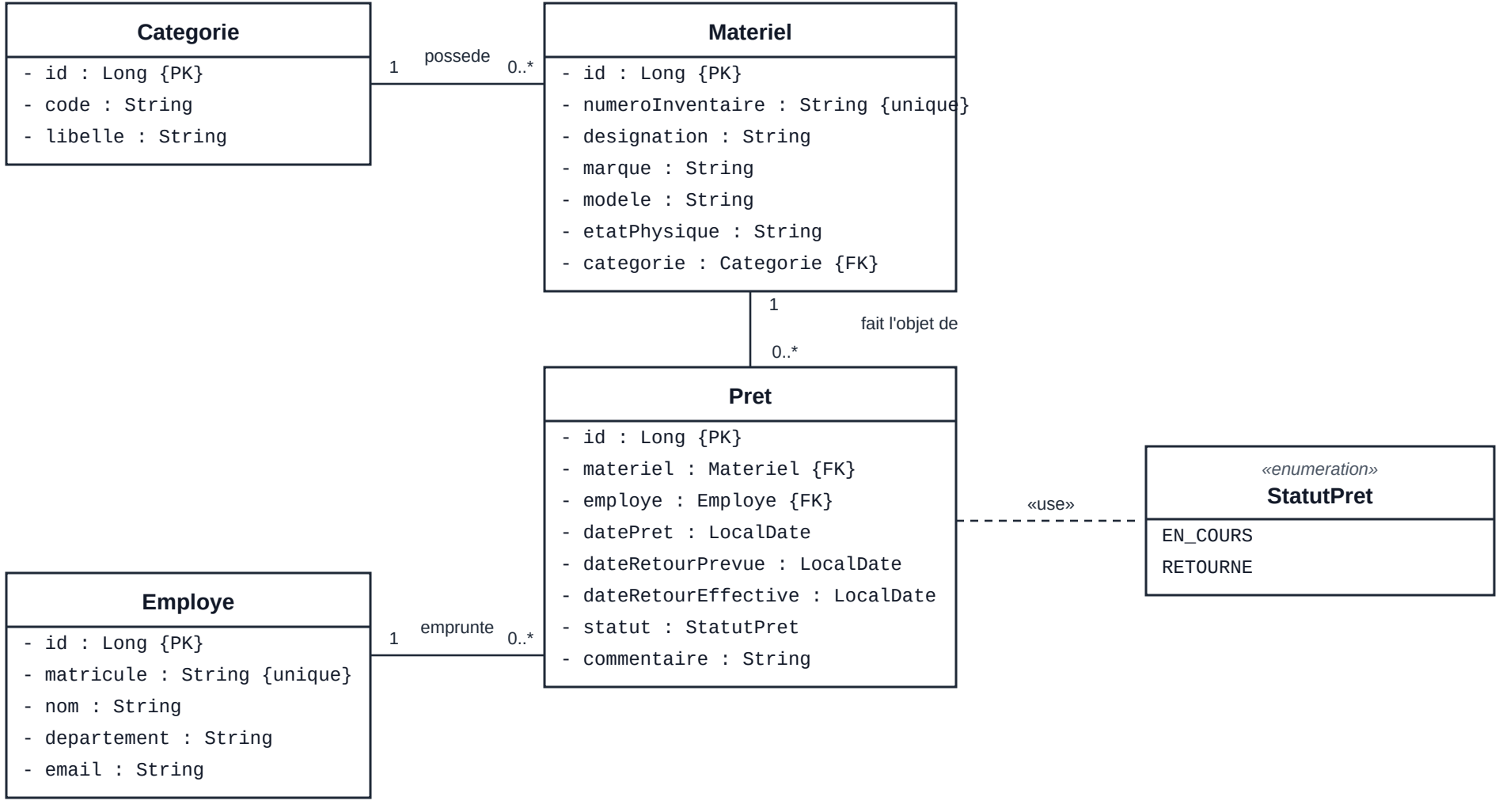
Le backend suit une architecture en couches classique, conforme au cahier des charges :

Couche	Rôle	Classes principales
Controller	Expose les endpoints REST	EmployeController, MaterielController, PretController, CategorieController
Service	Règles métier (disponibilité, validations, retard)	EmployeService, MaterielService, PretService, CategorieService
Repository	Accès aux données (Spring Data JPA)	EmployeRepository, MaterielRepository, PretRepository, CategorieRepository
Entity	Entités JPA / tables de la base	Employe, Materiel, Categorie, Pret, StatutPret
DTO	Séparation API / base de données	*Request / *Response pour chaque entité
Exception	Gestion centralisée des erreurs	GlobalExceptionHandler, BusinessRuleException, ResourceNotFoundException

3. Modèle de données

Quatre entités JPA, générées automatiquement en tables PostgreSQL au démarrage (`spring.jpa.hibernate.ddl-auto=update`). Le diagramme de classes complet suit en page suivante.

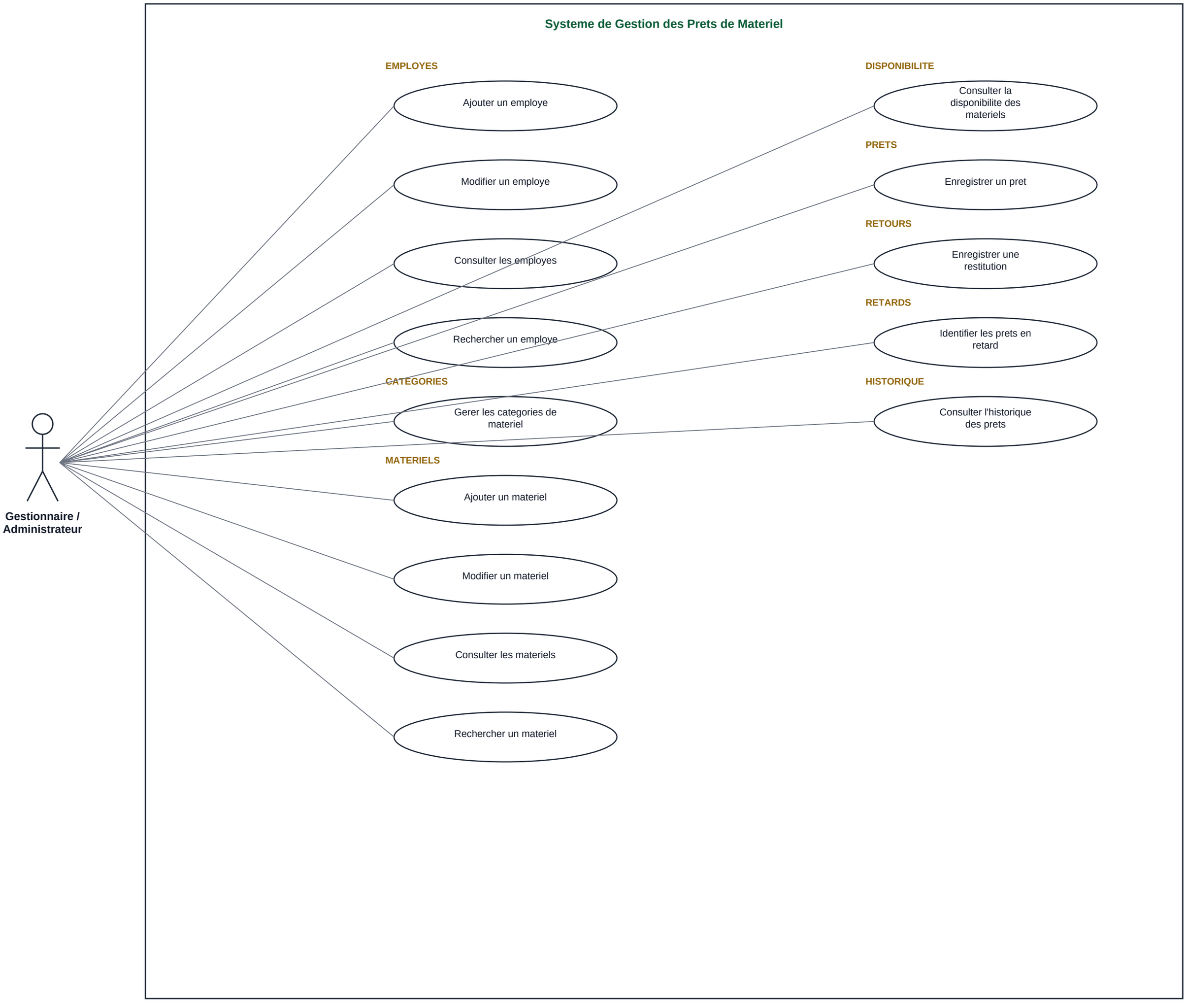
Diagramme de classes - Gestion des prets de materiel (BCM)



4. Cas d'utilisation

14 cas d'utilisation couvrant l'intégralité du tableau fonctionnel du sujet (section 3). Le diagramme complet suit en page suivante.

Diagramme de cas d'utilisation - Gestion des prets de materiel (BCM)



5. API REST — 26 endpoints implémentés

Employés

Méthode	Chemin	Description
GET	/api/employes	Liste des employés
GET	/api/employes/{id}	Détail d'un employé
GET	/api/employes/recherche?nom=	Recherche par nom
POST	/api/employes	Créer un employé
PUT	/api/employes/{id}	Modifier un employé
DELETE	/api/employes/{id}	Supprimer un employé

Catégories

Méthode	Chemin	Description
GET	/api/categories	Liste des catégories
GET	/api/categories/{id}	Détail d'une catégorie
POST	/api/categories	Créer une catégorie
PUT	/api/categories/{id}	Modifier une catégorie
DELETE	/api/categories/{id}	Supprimer une catégorie

Matériels

Méthode	Chemin	Description
GET	/api/materiels	Liste des matériels
GET	/api/materiels/{id}	Détail d'un matériel
GET	/api/materiels/disponibles	Matériels actuellement disponibles
GET	/api/materiels/recherche?designation=	Recherche par désignation
POST	/api/materiels	Créer un matériel
PUT	/api/materiels/{id}	Modifier un matériel
DELETE	/api/materiels/{id}	Supprimer un matériel

Prêts

Méthode	Chemin	Description
GET	/api/prets	Liste de tous les prêts
GET	/api/prets/{id}	Détail d'un prêt
GET	/api/prets/employe/{id}	Historique des prêts d'un employé
GET	/api/prets/materiel/{id}	Historique des prêts d'un matériel
GET	/api/prets/en-cours	Prêts actuellement en cours
GET	/api/prets/en-retard	Prêts en retard (calculé automatiquement)
POST	/api/prets	Enregistrer un nouveau prêt

POST	/api/prets/{id}/retour	Enregistrer une restitution

6. Règles métier implémentées

• Un matériel ne peut avoir qu'un seul prêt actif (EN_COURS) à la fois — vérifié avant toute création de prêt.

• Le matricule d'un employé et le numéro d'inventaire d'un matériel doivent être uniques.

• La date de retour prévue doit être postérieure ou égale à la date de prêt (double validation : service + contrainte Bean Validation).

• Un prêt déjà marqué RETOURNE ne peut pas être restitué une seconde fois.

• Le caractère « en retard » n'est jamais stocké : il est calculé à la volée (statut = EN_COURS et date prévue dépassée).

• Après restitution, un matériel redevient automatiquement disponible, sans champ dédié — sa disponibilité se déduit de l'absence de prêt EN_COURS le concernant.

• Toutes les erreurs métier renvoient un message clair et un code HTTP adapté (404, 409, 400), jamais une erreur technique brute (section 11 du cahier des charges).

7. Démonstration de bout en bout (résultats réels)

Scénario exécuté en direct contre le backend en cours d'exécution (pas de simulation), conforme au scénario attendu en section 16 du cahier des charges :

1. Consulter les matériels disponibles

```
GET /api/materiels/disponibles
```

→ 200 OK — liste des matériels sans prêt actif

2. Tenter de prêter un matériel déjà emprunté

```
POST /api/prets (materielId déjà EN_COURS)
```

→ 409 Conflict — {"message":"Ce matériel a déjà un prêt en cours (prêt #1)"}

3. Restituer un prêt en cours

```
POST /api/prets/3/retour
```

→ 200 OK — statut passe à "RETOURNE", dateRetourEffective renseignée automatiquement

4. Vérifier le retour à disponibilité

```
GET /api/materiels/disponibles
```

→ 200 OK — le matériel restitué réapparaît dans la liste

5. Consulter l'historique du matériel

```
GET /api/prets/materiel/4
```

→ 200 OK — prêt retourné visible dans l'historique

6. Rechercher un employé par nom

```
GET /api/employes/recherche?nom=sid
```

→ 200 OK — correspondance partielle, insensible à la casse

8. Intégration frontend ↔ backend

Le frontend (React + Vite) consomme directement l'API REST du backend. En développement, Vite proxifie les appels `/api/*` vers `http://localhost:8080`, évitant tout problème de CORS.

Page	Fonctionnalités connectées à l'API
Tableau de bord	Compteurs en temps réel (matériels, employés, prêts en cours, en retard) + liste des prêts en retard
Employés	Liste, création, modification, recherche
Matériels	Liste avec onglets Tous / Disponibles / Empruntés, création, modification
Prêts	Création d'un prêt, restitution, consultation des prêts en cours / en retard / historique
Catégories	Gestion des catégories de matériel

Identité visuelle : logo officiel de la BCM intégré, palette dérivée des couleurs réelles du logo (vert institutionnel #25A64B, or #F9EB16), mise en page sobre adaptée à un outil interne bancaire (pas de dégradés ni d'éléments décoratifs superflus).

9. Base de données et jeu de données de démonstration

PostgreSQL 18, base `gestion_prets`, schéma généré automatiquement par Hibernate. Un script `demo-data.sql` rejouable fournit un jeu de données cohérent :

Table	Contenu de démonstration
categories	5 catégories (Ordinateur portable, Écran, Téléphone, Vidéoprojecteur, Disque externe)
employes	5 employés de différents départements
materiels	8 matériels répartis sur toutes les catégories
prets	4 prêts couvrant tous les cas : en cours à temps, en cours en retard, retournés (historique multiple sur un même matériel)

10. Résultats des tests unitaires

Classe de test	Tests	Échecs	Erreurs
GestionPretsBcmApplicationTests	1	0	0
EmployeServiceTest	6	0	0
CategorieServiceTest	2	0	0
MaterielServiceTest	6	0	0
PretServiceTest	11	0	0
PretControllerTest	8	0	0
TOTAL	34	0	0

11. Conformité au cahier des charges

Exigence du sujet	Statut
Gestion des employés (CRUD + recherche)	Conforme
Gestion des catégories	Conforme
Gestion des matériels (CRUD + recherche + disponibilité)	Conforme
Enregistrement des prêts et restitutions	Conforme
Détection automatique des retards	Conforme
Historique des prêts (par employé et par matériel)	Conforme
API REST conforme à la section 8 du sujet	Conforme (26/26 endpoints)
Architecture Controller / Service / Repository / Entity / DTO / Exception	Conforme
Validations Jakarta sur les champs obligatoires	Conforme
Gestion centralisée des erreurs (messages compréhensibles)	Conforme
Collection Postman	Fournie (postman_collection.json)
Jeu de données de démonstration	Fourni (demo-data.sql)
Guide d'installation	Fourni (GUIDE_INSTALLATION.md)

Ce document a été généré à partir de l'état réel du projet (code source, base de données et tests exécutés), sans contenu fictif.